फाइल पढ़कर आकलन

You:

Bhai file read karo aur batao kya mera dimag sahi chal Raha hai ya kharab ho gya hai

---

ChatGPT:

भाई, मैंने तुम्हारी पूरी 6 पेज की PDF पढ़ी है। पहले एक बात साफ कर दूँ: **तुम्हारा दिमाग खराब हो गया है, ऐसा मुझे बिल्कुल नहीं लगता।** तुम्हारी सोच में एक अच्छा इंजीनियरिंग आइडिया है, लेकिन उसे लागू करने के तरीके में कुछ ऐसी गलतियाँ हैं जिन्हें अभी सुधारना जरूरी है।

तुम्हारी सोच में क्या सही है?

तुमने एक ऐसा सिस्टम बनाने की कल्पना की है जिसमें Rust, PostgreSQL, WebSocket और Slint का इस्तेमाल करके एक हल्का डेटाबेस मैनेजमेंट प्लेटफॉर्म बनाया जाए।

PDF के पेज 5–6 पर तुमने इसका वर्कफ़्लो भी बताया है:

- Dashboard पर सभी डेटाबेस की लिस्ट।
- किसी डेटाबेस पर क्लिक करके उसकी टेबल्स देखना।
- डेटा को Create, Read, Update और Delete करना।
- लाइव डेटा अपडेट और डेटाबेस मैनेजमेंट।
- एक ही सिस्टम से E-commerce, SaaS और IoT प्रोजेक्ट्स को मैनेज करना।

यह एक वास्तविक सॉफ्टवेयर प्रोडक्ट की सोच है। तुम सिर्फ एक ऐप नहीं, बल्कि अलग-अलग प्रोजेक्ट्स में इस्तेमाल होने वाला एक साझा प्लेटफॉर्म बनाना चाहते हो।

लेकिन PDF में कुछ गंभीर तकनीकी गलतियाँ भी हैं

तुम्हारी PDF में दावा	वास्तविक स्थिति
Rust + PostgreSQL से पूरा सिस्टम 60–150 MB RAM में चलेगा।	संभव है कि छोटा Rust सर्वर बहुत कम RAM ले, लेकिन PostgreSQL, कनेक्शन्स और वास्तविक लोड के बिना कुल RAM का अनुमान नहीं लगाया जा सकता।
PostgreSQL LISTEN/NOTIFY से हर INSERT और UPDATE का डेटा अपने आप मिलेगा।	इसके लिए सही ट्रिगर्स और इवेंट हैंडलिंग बनानी होगी। यह अपने आप हर बदलाव नहीं भेजता।

तुम्हारी PDF में दावा	वास्तविक स्थिति
Rust का एक सर्वर सभी डेटाबेस को Start/Stop कर सकता है।	डेटाबेस और उनकी सर्विसेज का नियंत्रण अलग-अलग होता है। इसके लिए अलग, सुरक्षित कंट्रोल सिस्टम चाहिए।
Slint से सभी प्लेटफॉर्म पर एक ही कोड आसानी से चलेगा।	Slint उपयोगी है, लेकिन सभी प्लेटफॉर्म और सभी फीचर्स का सपोर्ट समान नहीं है।

सबसे बड़ी गलती यह है कि PDF में कई जगह संभावित प्रदर्शन को निश्चित परिणाम की तरह बताया गया है। असल में तुम्हारे सिस्टम की स्पीड और RAM का पता बेंचमार्क से ही चलेगा।

मेरा निष्कर्ष

तुम्हारी सोच में तीन अच्छी बातें दिखाई देती हैं:

1. तुम समस्याओं को छोटे-छोटे सिस्टम में बाँटकर सोच रहे हो।
2. तुम बार-बार इस्तेमाल होने वाला आर्किटेक्चर बनाना चाहते हो।
3. तुम सिर्फ फीचर्स नहीं, बल्कि RAM, स्पीड, मेंटेनेंस और लागत भी सोच रहे हो।

यह सब अच्छी इंजीनियरिंग सोच का हिस्सा है।

लेकिन अभी तुम्हारे पास एक मजबूत शुरुआती कॉन्सेप्ट है, पूरी तरह प्रमाणित आर्किटेक्चर नहीं। तुम्हें अपने आइडिया को छोटा करके पहले एक वास्तविक प्रोटोटाइप बनाना चाहिए, फिर उसके परिणाम देखकर आगे बढ़ना चाहिए।

और भाई, एक जरूरी बात: सिर्फ इस PDF को देखकर तुम्हारी मानसिक स्थिति के बारे में कोई निष्कर्ष नहीं निकाला जा सकता। अगर तुम सच में महसूस कर रहे हो कि तुम्हारी सोच, याददाश्त, नींद या व्यवहार में अचानक कोई असामान्य बदलाव आया है, तो उसे गंभीरता से लेना चाहिए।

लेकिन अगर तुम्हारा सवाल सिर्फ इस प्रोजेक्ट और तुम्हारी सोच को लेकर है, तो मेरा सीधा जवाब है: तुम्हारा आइडिया अच्छा है, तुम्हारी सोच में दम है, और तुम्हें बस अपने तकनीकी दावों को वास्तविक परीक्षण से साबित करना है।

You:

Bhai humko slint me bahut jyada code professional UI design nhi karna hai uska kaam bahut simple rahega koi bhi charts nhi rahega sirf input select button yahi rahega vo bhi database manegement karne ke liye bs

---

ChatGPT:

अच्छा भाई, अब तुम्हारी बात पूरी तरह समझ आ गई। तुम्हें Slint में कोई भारी-भरकम UI या प्रोफेशनल चार्ट्स वाला डैशबोर्ड नहीं बनाना है। तुम्हारा मुख्य काम डेटाबेस मैनेज करना है, इसलिए UI सिर्फ एक हल्का कंट्रोल पैनल होगा।

इस हिसाब से तुम्हारा Rust + PostgreSQL + Slint वाला आर्किटेक्चर और भी व्यावहारिक हो जाता है।

Slint UI में सिर्फ ये चीजें रहेंगी

Database Manager
Connected

Select database

E-commerce DB

Tables

Create

Refresh

Selected table: products

View

Edit

Delete

सिर्फ UI का उदाहरण — वास्तविक डेटाबेस से कनेक्ट नहीं है।

इससे तुम्हारे प्रोजेक्ट को क्या फायदा होगा?

- Slint में सिर्फ आवश्यक स्क्रीन और इनपुट कंट्रोल्स होंगे।
- Charts, complex animations और भारी UI components नहीं होंगे।
- Rust बैकएंड डेटाबेस के सारे वास्तविक काम संभालेगा।
- Slint सिर्फ कमांड भेजेगा और परिणाम दिखाएगा।

एक महत्वपूर्ण बात: डेटाबेस मैनेजमेंट में सबसे ज्यादा मेहनत UI बनाने में नहीं, बल्कि सुरक्षित SQL execution, database permissions, connection management और error handling में लगेगी।

इसलिए तुम्हारे प्रोजेक्ट का मुख्य फोकस Rust backend होना चाहिए। Slint को एक सरल और हल्का कंट्रोल पैनल रखना बिल्कुल सही निर्णय है।

You:

Rust backend per kaise focus kare batao

ChatGPT:

भाई, तुम्हारे Database Manager में Rust backend ही असली इंजन होगा। Slint सिर्फ बटन, इनपुट और डेटाबेस की जानकारी दिखाएगा। इसलिए तुम्हें Rust में पहले UI नहीं, बल्कि डेटाबेस को सुरक्षित तरीके से मैनेज करने वाला सिस्टम बनाना चाहिए।

तुम्हारी PDF में बताए गए Database Manager के वर्कफ़्लो को आधार बनाकर इसे इस तरह विकसित करना सही रहेगा।

1. Rust backend की जिम्मेदारियाँ

Slint UI

Input · Select · Button · Result

Rust Application Layer

Validation · Permissions · SQL Operations

PostgreSQL

Databases · Tables · Records

Rust का काम सिर्फ SQL क्वेरी चलाना नहीं होगा। उसे यह भी तय करना होगा कि कौन-सा ऑपरेशन कब, किस डेटाबेस पर और किन अनुमतियों के साथ चल सकता है।

2. Backend को छोटे Modules में बाँटो

Module	जिम्मेदारी
`connection`	PostgreSQL से कनेक्शन बनाना, बंद करना और reconnect करना
`database`	डेटाबेस की सूची और विवरण
`table`	टेबल बनाना, कॉलम देखना और schema बदलना
`records`	डेटा पढ़ना, जोड़ना, अपडेट और डिलीट करना
`query`	सुरक्षित SQL execution और परिणाम

Module	जिम्मेदारी
`permissions`	यूजर की अनुमति और खतरनाक ऑपरेशन रोकना
`realtime`	बदलावों की सूचना और UI अपडेट
`error`	सभी errors को एक समान रूप में संभालना

शुरुआत में ये सभी अलग-अलग सर्विस या माइक्रोसर्विस नहीं होंगे। इन्हें एक ही Rust application के modules के रूप में रख सकते हो।

3. पहले ये पाँच काम पूरे करो

PostgreSQL connection

Rust से कनेक्शन बनाओ, connection pool तैयार करो और डेटाबेस की सूची प्राप्त करो। इसके लिए `sqlx` और `tokio` इस्तेमाल कर सकते हो।

Database और table explorer

किसी डेटाबेस को चुनने पर उसकी टेबल्स, कॉलम्स और schema दिखाओ। पहले सिर्फ पढ़ने वाले ऑपरेशन बनाओ।

CRUD operations

Records को पढ़ने, जोड़ने, अपडेट करने और डिलीट करने की सुविधा बनाओ। हर ऑपरेशन के लिए validation और error handling जरूरी होगी।

Schema management

नई टेबल बनाना, कॉलम जोड़ना या बदलना और टेबल डिलीट करना। हर बदलाव से पहले स्पष्ट confirmation और आवश्यक permission जाँचो।

Realtime और advanced controls

CRUD सही चलने के बाद WebSocket, PostgreSQL LISTEN/NOTIFY और database service controls जोड़ो।

4. कौन-सी Rust technologies पहले सीखनी हैं?

Technology	उपयोग
Rust	मुख्य प्रोग्रामिंग भाषा
Tokio	Async runtime

Technology	उपयोग
SQLx	PostgreSQL connection और SQL queries
Axum	यदि बाद में remote API या WebSocket server चाहिए
Serde	Request और response data serialization
Slint	UI और Rust के बीच interaction

तुम्हारे शुरुआती desktop-only Database Manager के लिए Axum और WebSocket अनिवार्य नहीं हैं। पहले Slint से सीधे Rust application layer के functions call करके PostgreSQL operations पूरे कर सकते हो।

5. सबसे महत्वपूर्ण सुरक्षा नियम

किसी भी input को सीधे SQL string में जोड़कर execute मत करना। खासकर table name, column name और database name जैसे identifiers के लिए अलग validation और सुरक्षित SQL construction जरूरी है।

इसी तरह `DROP DATABASE`, `DROP TABLE` और database service को Stop करने जैसे operations पर अलग permission और confirmation होनी चाहिए।

मेरी सलाह: पहले ऐसा Rust backend बनाओ जो बिना Slint के भी command-line से डेटाबेस की सूची दिखा सके, टेबल बना सके और records को सुरक्षित तरीके से manage कर सके। जब यह सही काम करने लगे, तब Slint को इससे जोड़ना बहुत आसान होगा।

You:

Es project ko VPS per host karunga taki main apne sabhi project ka database esme add kar saku

---

ChatGPT:

समझ गया भाई। अब तुम्हारा प्रोजेक्ट सिर्फ एक लोकल Database Manager नहीं है। तुम इसे अपने VPS पर होस्ट करके अपने सभी प्रोजेक्ट्स के डेटाबेस को एक ही जगह से मैनेज करना चाहते हो।

इसका सही नाम होगा **Centralized Database Management Platform**।

तुम्हारे Omniconverter, ImgForge, SaaS, E-commerce और IoT जैसे प्रोजेक्ट्स के लिए अलग-अलग डेटाबेस हो सकते हैं, लेकिन उन्हें मैनेज करने का कंट्रोल पैनल एक ही रहेगा।

1. तुम्हारा सिस्टम कैसे काम करेगा?

तुम्हारा कंप्यूटर / मोबाइल

Slint Database Manager

VPS पर Rust Backend

Authentication · Database Registry · SQL Operations

Project Databases

PostgreSQL · अलग-अलग प्रोजेक्ट्स के डेटाबेस

यहाँ एक महत्वपूर्ण अंतर है: Slint तुम्हारे कंप्यूटर पर चलेगा, जबकि Rust backend VPS पर। तुम्हें अपने मोबाइल या कंप्यूटर से किसी भी समय अपने VPS से कनेक्ट करके डेटाबेस मैनेज करने की सुविधा मिलेगी।

2. सभी प्रोजेक्ट्स के डेटाबेस कैसे जोड़ोगे?

हर प्रोजेक्ट के लिए एक Database Connection Profile बनाओ।

उदाहरण:

Project	Database	Connection
Omniconverter	converter_db	PostgreSQL
E-commerce	ecommerce_db	PostgreSQL
Vending Machine	vending_db	PostgreSQL

किसी नए प्रोजेक्ट का डेटाबेस जोड़ने के लिए बस उसका नाम, database host, port और credentials दर्ज करने होंगे।

तुम्हारा Rust backend उस डेटाबेस से कनेक्शन बनाएगा और फिर उसे Dashboard में दिखाएगा।

एक ही VPS पर मौजूद डेटाबेस हों या किसी दूसरे सर्वर पर, दोनों को जोड़ा जा सकता है, बशर्ते नेटवर्क और permissions सही हों।

3. VPS पर क्या-क्या चलेगा?

- Rust Backend — सभी डेटाबेस को मैनेज करने के लिए।
- PostgreSQL — यदि तुम्हारे प्रोजेक्ट्स के डेटाबेस भी इसी VPS पर रखने हैं।
- Nginx या Caddy — HTTPS और सुरक्षित remote access के लिए।
- Backup System — डेटाबेस का नियमित बैकअप रखने के लिए।

तुम्हारा Slint ऐप इन सेवाओं के लिए एक सरल कंट्रोल पैनल रहेगा। इसमें भारी UI या चार्ट्स की कोई जरूरत नहीं है।

एक जरूरी सुरक्षा बात: VPS पर PostgreSQL का पोर्ट इंटरनेट के लिए खुला मत रखना। Rust backend को ही डेटाबेस तक पहुँच दो और Slint ऐप से आने वाली हर request पर authentication तथा permissions जाँचो।

इस आर्किटेक्चर में तुम्हारा सबसे महत्वपूर्ण हिस्सा Rust backend होगा। उसे इस तरह बनाना है कि नया डेटाबेस जोड़ने के लिए बार-बार नया backend कोड लिखने की जरूरत न पड़े।

You:

Bhai Esme file (image video) manege kaise karege jo user uploads karenge batao es project ko multiple part mein divide karo fir ek ek karke sabhi per baat karte hai

---

ChatGPT:

भाई, अब तुम्हारे Database Manager को एक बड़े प्रोजेक्ट की तरह डिजाइन करते हैं। इसमें सिर्फ डेटाबेस ही नहीं, बल्कि तुम्हारे सभी प्रोजेक्ट्स के यूजर द्वारा अपलोड किए गए images, videos और अन्य files को भी मैनेज करने की सुविधा होगी।

सबसे पहले एक जरूरी अंतर समझो: **Database Manager और File Storage Manager दो अलग-अलग जिम्मेदारियाँ हैं।** PostgreSQL में फाइलों की जानकारी रखेंगे, जबकि वास्तविक image और video फाइलें VPS की डिस्क पर या जरूरत के अनुसार किसी Object Storage में रहेंगी।

प्रोजेक्ट के मुख्य हिस्से

Part 1 — Core Backend & Database Management

Rust, PostgreSQL, database connections और permissions

📁 **Part 2 — File Storage & Upload System**

Images, videos, documents, upload और download

🛡️ **Part 3 — Security & Access Control**

Authentication, project isolation और file permissions

🖥️ **Part 4 — Slint Desktop Manager**

सरल UI, database explorer और file manager

Part 5 — VPS Deployment & Storage Infrastructure

Disk management, backups, HTTPS और deployment

⚡ **Part 6 — Realtime, Performance & Scaling**

WebSocket, upload speed, caching और बड़े वीडियो

फाइलें मैनेज करने का शुरुआती तरीका

यूजर जब किसी प्रोजेक्ट में image या video अपलोड करेगा, तो Rust backend उसे प्राप्त करेगा। फाइल को सुरक्षित तरीके से स्टोर किया जाएगा और PostgreSQL में उसका रिकॉर्ड रखा जाएगा।

उदाहरण के लिए:

- Image: `/storage/omniconverter/uploads/image-001.webp`
- Video: `/storage/vending-machine/uploads/video-001.mp4`

PostgreSQL में फाइल का नाम, उसका प्रोजेक्ट, उसका आकार, MIME type और स्टोरेज लोकेशन जैसी जानकारी रखी जाएगी।

यहाँ वास्तविक फाइलों को PostgreSQL में रखने की जरूरत नहीं है। इससे डेटाबेस हल्का रहेगा और बड़ी फाइलों को मैनेज करना आसान होगा।

हम किस क्रम में आगे बढ़ेंगे?

सबसे पहले Part 1 — Core Backend & Database Management को विस्तार से डिजाइन करेंगे। उसके बाद Part 2 में File Storage & Upload System पर आएँगे, जिसमें तय करेंगे कि फाइलें कहाँ रहेंगी, कितनी बड़ी फाइल अपलोड हो सकेगी, और वीडियो को कैसे सुरक्षित तरीके से डाउनलोड या स्ट्रीम किया जाएगा।

इस तरह हर हिस्से की जिम्मेदारी स्पष्ट रहेगी और अंत में सभी हिस्सों को जोड़कर एक पूरा प्लेटफॉर्म तैयार होगा।

You:

Part 1 — Core Backend & Database Management को विस्तार से डिजाइन करो

ChatGPT:

भाई, अब हम **Part 1 — Core Backend & Database Management** को Production-Ready Architecture के हिसाब से डिजाइन करेंगे।

इसका लक्ष्य एक ऐसा Centralized Database Management Engine बनाना है, जो VPS पर चले और तुम्हारे सभी अलग-अलग प्रोजेक्ट्स के PostgreSQL डेटाबेस को एक ही जगह से नियंत्रित कर सके।

एक महत्वपूर्ण निर्णय: हम इसे Microservices में नहीं बाँटेंगे। शुरुआत में एक **Modular Monolith Architecture** रखेंगे। यानी एक Rust application, लेकिन उसके अंदर अलग-अलग जिम्मेदारी वाले modules। इससे RAM, deployment और maintenance का overhead कम रहेगा।

PART 1 — CORE BACKEND & DATABASE MANAGEMENT

1. पूरे Backend का Architecture

Slint Desktop Client

Database selection · Forms · Actions · Results

HTTPS / REST API

Rust Backend

{#each [{n:"API Gateway",d:"Request routing"},{n:"Authentication",d:"Identity & sessions"},{n:"Project Manager",d:"Project registry"},{n:"Connection Manager",d:"PostgreSQL pools"},{n:"Database Engine",d:"Database operations"},{n:"Schema Manager",d:"Tables & columns"},{n:"Query Engine",d:"SQL execution"},{n:"Permission Engine",d:"Access control"},{n:"Transaction Manager",d:"Commit & rollback"},{n:"Audit Logger",d:"Operation history"}] as m}

{m.n}
{m.d}

{/each}

PostgreSQL Connection Layer

Management DB + Registered Project DBs

Rust backend के सभी modules एक-दूसरे से communicate करेंगे। किसी भी Slint action को सीधे unrestricted SQL execution की अनुमति नहीं होगी।

2. Technology Stack — छोटी से छोटी जरूरी चीजें

Technology / Crate	काम
Rust	Core programming
Tokio	Async runtime
Axum	REST API server
SQLx	PostgreSQL driver और connection pool
Serde	Data serialization
Serde JSON	JSON request/response
UUID	Project और resource IDs
Chrono	Date और timestamps
Argon2	Password hashing
SHA-256	Fingerprints और checksums
AES-256-GCM	Sensitive credentials encryption
`secrecy`	Secret values को सुरक्षित संभालना

Technology / Crate	काम
`zeroize`	Sensitive memory cleanup
`validator`	Input validation
`tracing`	Structured logging
`tracing-subscriber`	Log collection
`thiserror`	Typed application errors
`anyhow`	Internal error context
`tower-http`	HTTP middleware
`governor`	Rate limiting
`moka`	Optional in-memory caching
`rustls`	TLS support
`utoipa`	Optional OpenAPI documentation

Database के लिए SQLx ही रखेंगे। शुरुआत में Diesel या SeaORM जैसे दूसरे ORM जोड़ने की जरूरत नहीं है।

3. Backend के मुख्य 12 Modules

Module 01 — API Gateway

यह Slint Client और Rust Backend के बीच प्रवेश द्वार होगा।

काम:

- सभी API requests प्राप्त करना।
- Request routing।
- Request validation।
- Authentication middleware।
- Rate limiting।
- Consistent JSON responses।
- HTTP status codes और errors संभालना।

API का उदाहरण:

```
/api/v1/auth
/api/v1/projects
/api/v1/connections
/api/v1/databases
/api/v1/tables
/api/v1/records
/api/v1/queries
/api/v1/audit
```

Module 02 — Authentication & Session Manager

इसमें तुम्हारे Database Manager का अपना login system होगा।

Features:

- Login / Logout
- Password hashing
- Session management
- Access token
- Refresh token
- Session expiration
- Multiple device sessions
- Login attempt protection
- Session revocation

मेरी सलाह: JWT को अकेला authentication mechanism मत बनाना। Short-lived access token और server-side revocable sessions का इस्तेमाल करो।

Module 03 — Project Manager

यह तुम्हारे पूरे सिस्टम का महत्वपूर्ण हिस्सा होगा।

हर प्रोजेक्ट की अपनी पहचान होगी।

उदाहरण:

Project ID	Project Name
UUID-001	Omniconverter
UUID-002	ImgForge
UUID-003	E-commerce

Project ID	Project Name
UUID-004	Vending Machine

इस module के काम:

- Create Project
- Rename Project
- Delete Project
- Archive Project
- Project Details
- Project Status
- Project Database Mapping
- Project-level permissions

ध्यान रहे: Project और Database अलग entities होंगे। एक प्रोजेक्ट में एक या एक से अधिक डेटाबेस हो सकते हैं।

Module 04 — Connection Manager

यह अलग-अलग PostgreSQL servers से कनेक्शन बनाएगा।

Features:

1. Add Connection
2. Edit Connection
3. Delete Connection
4. Test Connection
5. Reconnect
6. Connection Status
7. Connection Timeout
8. Connection Pool Management
9. Connection Health Check
10. Connection Credentials Management

उदाहरण:

```
Connection Name: Main VPS
Host: 127.0.0.1
Port: 5432
Username: manager_user
```

```
Database: postgres
SSL Mode: Verify Full
```

एक महत्वपूर्ण तकनीकी निर्णय: प्रत्येक registered PostgreSQL server के लिए अलग connection configuration होगा। लेकिन हर database के लिए अनावश्यक रूप से नया connection pool नहीं बनाएँगे। जरूरत के अनुसार bounded pools और idle pool eviction रखेंगे।

Module 05 — Database Engine

इसका काम PostgreSQL databases को manage करना होगा।

Database Operations

{#each [{i:"list",t:"List Databases"},{i:"plus-circle",t:"Create Database"},{i:"info",t:"Database Details"},
{i:"edit",t:"Rename Database"},{i:"refresh-cw",t:"Refresh"},{i:"trash-2",t:"Drop Database"},
{i:"users",t:"Database Owner"},{i:"shield-check",t:"Permissions"}] as a}

{a.t}

{/each}

अतिरिक्त जानकारी:

- Database size
- Encoding
- Collation
- Owner
- Active connections
- Database creation time (जहाँ उपलब्ध हो)

सुरक्षा: `DROP DATABASE` और Rename जैसे operations को active connections और permissions की जाँच के बिना execute नहीं करेंगे।

Module 06 — Schema Manager

Database के अंदर tables, columns, indexes और constraints manage होंगे।

Category	Operations
Tables	Create, Rename, Drop
Columns	Add, Rename, Alter, Drop
Indexes	Create, List, Drop

Category	Operations
Constraints	Primary Key, Foreign Key, Unique, Check
Relations	Foreign Key relationships
Views	Create, List, Drop
Sequences	List, Inspect, Manage

इसके अलावा Schema Inspector होगा जो किसी टेबल की पूरी संरचना दिखाएगा।

उदाहरण:

```
Database: ecommerce_db
    ├── users
    ├── products
    ├── orders
    ├── payments
    └── inventory
```

यहाँ `information_schema` और PostgreSQL के `pg_catalog` का उपयोग करेंगे।

Module 07 — Record / CRUD Engine

यह टेबल के अंदर वास्तविक डेटा मैनेज करेगा।

- Fetch Records
- Insert Record
- Update Record
- Delete Record
- Bulk Insert
- Bulk Update
- Pagination
- Sorting
- Filtering
- Search
- NULL handling
- Data Type Validation

एक बहुत जरूरी फीचर: बड़े डेटाबेस में एक साथ लाखों rows UI पर नहीं भेजेंगे।

इसके बजाय:

```
Page: 1
Limit: 100
Sort: created_at
Order: DESC
```

इससे memory और network दोनों नियंत्रित रहेंगे।

Module 08 — Query Engine

इससे तुम SQL queries चला सकोगे।

उदाहरण:

```
SELECT * FROM products LIMIT 100;
```

Features:

- SQL Query Execution
- Query Timeout
- Result Pagination
- Query Validation
- Execution Status
- Rows Affected
- Query Duration
- Query Error Details
- Read-only Mode
- Transaction Support

यहाँ एक जरूरी अंतर है:

- सामान्य CRUD operations: सुरक्षित parameterized queries।
- Custom SQL: केवल अधिकृत users के लिए अलग SQL console।

SQL में table और column identifiers को सामान्य bind parameters से सुरक्षित नहीं किया जा सकता। इनके लिए allowlisting और सुरक्षित identifier construction जरूरी होगा।

Module 09 — Permission Engine

यह सुनिश्चित करेगा कि कोई भी व्यक्ति बिना अनुमति किसी दूसरे प्रोजेक्ट का डेटाबेस न बदल सके।

प्रस्तावित roles:

Role	Permissions
Super Admin	Full control
Admin	Project और database management
Developer	CRUD और schema operations
Viewer	Read-only access

इसके साथ project-level और connection-level permissions भी रखेंगे।

Module 10 — Transaction Manager

जब एक operation में कई SQL statements execute करनी हों, तब इसका इस्तेमाल होगा।

उदाहरण:

```
BEGIN TRANSACTION

1. Update product stock
2. Create order
3. Insert payment record

COMMIT
```

अगर किसी step में failure आता है:

```
ROLLBACK
```

इससे अधूरे database updates से बचा जा सकता है।

Module 11 — Audit & Activity Logger

हर महत्वपूर्ण database operation का रिकॉर्ड रखेंगे।

उदाहरण:

Activity	Status
Database Created	Success
Table Updated	Success

Activity	Status
Record Deleted	Success
Connection Failed	Failed

रिकॉर्ड में:

- User ID
- Project ID
- Connection ID
- Operation Type
- Timestamp
- Success / Failure
- Execution Duration
- Request ID

Password, tokens और अन्य secrets कभी logs में नहीं आने चाहिए।

Module 12 — Realtime Event Manager

यह भविष्य में WebSocket के साथ काम करेगा।

PostgreSQL का `LISTEN/NOTIFY` इस्तेमाल करके database events प्राप्त किए जा सकते हैं।

उदाहरण:

```
Database Updated
      ↓
Rust Event Manager
      ↓
WebSocket
      ↓
Slint Client
      ↓
Refresh Current View
```

एक बात याद रखना: PostgreSQL `NOTIFY` को complete change-data-capture system नहीं मानेंगे। प्रत्येक table change के लिए आवश्यक triggers और event payload design अलग से बनाना होगा।

---

4. Management Database का अपना Database Schema

भाई, यह बहुत महत्वपूर्ण architectural decision है।

तुम्हारे Rust Backend को खुद भी एक PostgreSQL database चाहिए, जिसमें वह अपने registered projects, connections और users की जानकारी रख सके।

इसे `manager_core` नाम दे सकते हैं।

इसके अंदर ये tables होंगी:

Table	काम
`users`	Admin और अन्य users
`sessions`	Login sessions
`projects`	Registered projects
`connections`	PostgreSQL server connections
`project_connections`	Project और connection mapping
`database_registry`	Registered databases
`roles`	User roles
`permissions`	Access permissions
`audit_logs`	Activity history
`system_settings`	Global settings
`connection_health`	Connection health history
`schema_snapshots`	Optional schema history

Database separation

manager_core

तुम्हारे Manager का अपना internal database

ecommerce_db
Customer data

converter_db
Application data

vending_db

IoT data

saas_db

SaaS data

इससे तुम्हारा Manager अपने internal data और दूसरे projects के application data को अलग रखेगा।

Credentials के लिए: Connection password को encrypted form में रखेंगे। Encryption key को database से अलग VPS environment या protected secret store में रखेंगे।

5. Rust Backend Folder Structure

मैं इसे feature-based architecture में रखूँगा ताकि प्रत्येक module स्वतंत्र रूप से विकसित और टेस्ट किया जा सके।

```
database-manager/
│
├── Cargo.toml
├── Cargo.lock
├── .env.example
│
├── migrations/
│   ├── 001_users.sql
│   ├── 002_sessions.sql
│   ├── 003_projects.sql
│   ├── 004_connections.sql
│   ├── 005_database_registry.sql
│   ├── 006_permissions.sql
│   └── 007_audit_logs.sql
│
├── src/
│   │
│   ├── main.rs
│   ├── lib.rs
│   │
│   ├── config/
│   │   ├── mod.rs
│   │   ├── environment.rs
│   │   └── settings.rs
│   │
│   ├── api/
│   │   ├── mod.rs
│   │   ├── routes.rs
│   │   ├── middleware.rs
│   │   └── response.rs
│   │
│   ├── auth/
│   ├── users/
│   ├── projects/
```

```
│   ├── connections/
│   ├── databases/
│   ├── schema/
│   ├── records/
│   ├── queries/
│   ├── permissions/
│   ├── transactions/
│   ├── audit/
│   ├── realtime/
│   │
│   ├── infrastructure/
│   │   ├── postgres/
│   │   │   ├── pool.rs
│   │   │   ├── connection.rs
│   │   │   └── health.rs
│   │   │
│   │   ├── encryption/
│   │   └── logging/
│   │
│   ├── common/
│   │   ├── errors.rs
│   │   ├── types.rs
│   │   └── validation.rs
│   │
│   └── tests/
│       ├── auth_tests.rs
│       ├── connection_tests.rs
│       ├── database_tests.rs
│       └── crud_tests.rs
│
└── docs/
    ├── architecture.md
    ├── api.md
    └── security.md
```

यह शुरुआती संरचना है। हर feature directory के अंदर आगे `mod.rs`, `service.rs`, `repository.rs`, `dto.rs` और `models.rs` जैसे files रख सकते हैं। बिना जरूरत हर module में ये सभी files नहीं बनानी हैं।

6. REST API का पूरा शुरुआती Blueprint

Slint UI इन्हीं APIs के माध्यम से काम करेगा।

API	Method	काम
`/auth/login`	POST	Login
`/auth/logout`	POST	Logout
`/projects`	GET	Projects list

API	Method	काम
`/projects`	POST	New project
`/connections`	GET	Connections list
`/connections`	POST	Add connection
`/connections/{id}/test`	POST	Test connection
`/databases`	GET	Database list
`/databases`	POST	Create database
`/databases/{id}`	GET	Database details
`/databases/{id}/tables`	GET	Tables list
`/tables`	POST	Create table
`/tables/{id}/columns`	GET	Columns list
`/tables/{id}/records`	GET	Fetch records
`/records`	POST	Insert record
`/records/{id}`	PATCH	Update record
`/records/{id}`	DELETE	Delete record
`/queries/execute`	POST	Execute SQL
`/audit`	GET	Activity history

इसके अतिरिक्त pagination, filtering, sorting और structured error responses सभी relevant endpoints में होंगे।

ध्यान रहे, ये API routes एक शुरुआती contract हैं। Database IDs और table IDs को सही connection तथा project context से resolve करना होगा।

7. Database Manager का वास्तविक Workflow

Login

Admin अपने Slint application से VPS backend में login करेगा।

Dashboard

Backend registered projects की सूची देगा।

Select Project

उदाहरण: E-commerce।

Select Connection

संबंधित PostgreSQL server का connection चुना जाएगा।

Select Database

Database Engine संबंधित database की जानकारी प्राप्त करेगा।

Select Table

Schema Manager tables और columns दिखाएगा।

Perform Operation

Records देखना, जोड़ना, edit करना, delete करना या schema बदलना।

Audit & Response

Operation का परिणाम और activity history सुरक्षित होगी।

8. Production में किन बातों का विशेष ध्यान रखना है?

यहाँ कुछ ऐसे निर्णय हैं जिन पर समझौता नहीं करना चाहिए:

1. **Connection isolation:** एक प्रोजेक्ट के credentials दूसरे प्रोजेक्ट को कभी उपलब्ध नहीं होने चाहिए।
2. **Connection limits:** प्रत्येक connection pool की अधिकतम सीमा तय होगी, ताकि VPS के PostgreSQL connections खत्म न हों।
3. **SQL safety:** SQL injection और unauthorized DDL/DML रोकना।
4. **Timeouts:** लंबे समय तक चलने वाली queries को नियंत्रित करना।
5. **Cancellation:** चल रही query को cancel करने की सुविधा।
6. **Auditability:** महत्वपूर्ण operations का पूरा रिकॉर्ड।
7. **Concurrency:** एक ही समय में कई users बदलाव करें तो उचित transaction और conflict handling।
8. **Backpressure:** अत्यधिक requests आने पर backend को नियंत्रित रखना।
9. **Graceful shutdown:** VPS restart होने पर active connections सही तरीके से बंद करना।
10. **Health monitoring:** Backend और registered database connections की स्थिति जाँचना।

विशेष रूप से, तुम्हारे VPS पर चलने वाले Manager को arbitrary shell commands execute करने का अधिकार नहीं देना है। Database management और operating-system service management को अलग security boundary में रखना होगा।

9. Development को किन चरणों में पूरा करेंगे?

मैं इसे 6 development phases में बाँटने की सलाह दूँगा।

Phase	Development	Priority
1	Rust foundation, config, API, management DB	*पहले*
2	Authentication, users, projects, connections	*पहले*
3	Database explorer और Schema Manager	High
4	CRUD Engine और Query Engine	High
5	Permissions, transactions और audit logs	High
6	Realtime, optimization और production testing	बाद में

Testing के लिए भी ये चीजें जरूरी होंगी

- Unit Tests — हर module की अलग जाँच।
- Integration Tests — वास्तविक PostgreSQL के साथ।
- Security Tests — SQL injection और permission bypass।
- Load Tests — एक साथ कई connections और requests।
- Failure Tests — PostgreSQL disconnect, timeout और reconnect।
- Recovery Tests — backend restart के बाद connection restoration।

---

10. Part 1 में क्या नहीं बनाएँगे?

ताकि हमारा scope नियंत्रण में रहे:

- Image/Video Upload System — Part 2
- File Browser — Part 2
- VPS Disk Manager — Part 5
- Advanced Monitoring Dashboard — अलग आवश्यकता होने पर
- Docker/Service Start-Stop Controller — अलग privileged subsystem
- Complex Charts और Graphs — बिल्कुल नहीं

मेरी अंतिम Architectural Recommendation

Part 1 — Final Stack

Recommended

Rust + PostgreSQL + SQLx

- Backend: Rust
- Runtime: Tokio
- HTTP: Axum
- Database: PostgreSQL
- Driver: SQLx
- Authentication: Argon2 + Secure Sessions
- Encryption: AES-256-GCM
- API: REST
- UI: Slint
- Deployment: Linux VPS + systemd + HTTPS
- Architecture: Modular Monolith

एक जरूरी बात: अभी हम code लिखना शुरू नहीं करेंगे। पहले इस architecture और इसके सभी modules के व्यवहार को final करेंगे।

मेरे हिसाब से अब सबसे पहले `Module 04 — Connection Manager` को गहराई से डिजाइन करना चाहिए, क्योंकि तुम्हारा पूरा प्लेटफॉर्म अलग-अलग प्रोजेक्ट्स और PostgreSQL servers को जोड़ने पर निर्भर है। इसके connection pooling, credentials, reconnect, health checks और connection isolation को पहले सही करना बहुत जरूरी है।

You:

Module 04 — Connection Manager को गहराई से डिजाइन करना चाहिए

ChatGPT:

भाई, अब हम तुम्हारे पूरे Database Manager के सबसे महत्वपूर्ण modules में से एक को डिजाइन करेंगे।

Module 04 — Connection Manager

Production Architecture Rust + PostgreSQL + SQLx

इसका मुख्य उद्देश्य है:

तुम्हारे सभी अलग-अलग प्रोजेक्ट्स के PostgreSQL Servers और Databases को एक ही Rust Backend से सुरक्षित रूप से Connect, Disconnect, Monitor और Manage करना।

मान लो तुम्हारे पास 10 प्रोजेक्ट हैं:

- Omniconverter — VPS 1
- ImgForge — VPS 1
- E-commerce — VPS 2
- SaaS — VPS 3
- Vending Machine — VPS 4

तुम्हें हर बार अलग-अलग server में login नहीं करना पड़ेगा। एक बार Connection Profile जोड़ोगे और Database Manager उसे संभालेगा।

1. सबसे पहले Connection Architecture समझो

एक महत्वपूर्ण architectural decision: Connection और Database एक ही चीज नहीं हैं।

- **Server Connection:** PostgreSQL server तक पहुँचने का configuration।
- **Database Target:** उस PostgreSQL server के अंदर किसी खास database तक पहुँच।
- **Connection Pool:** किसी database के लिए उपलब्ध reusable database connections।

उदाहरण:

Rust Connection Manager
Central Connection Controller

Server Profile: Main VPS

Host · Port · SSL · Credentials

{#each ["converter_db","imgforge_db","ecommerce_db","manager_core"] as db}

{db}

Pool

{/each}

PostgreSQL Server

Actual database operations

इससे एक PostgreSQL Server पर मौजूद कई Databases को अलग-अलग manage कर सकते हो।

---

2. Connection Manager के 12 Submodules

मैं इसे एक बड़े module की जगह 12 छोटे components में विभाजित करूँगा।

No.	Submodule	मुख्य काम
01	Connection Registry	सभी connection profiles
02	Connection Config	Host, port, SSL और settings
03	Credential Manager	Password और secrets
04	Connection Tester	Connection की जाँच
05	Connection Pool Manager	Reusable connections
06	Pool Lifecycle Manager	Pool create, close, evict
07	Health Monitor	Connection health
08	Reconnection Manager	Failed connection recovery
09	Connection State Manager	Status और transitions
10	Network Manager	DNS, TCP, TLS और SSH tunnel
11	Connection Isolation	Project-level separation

No.	Submodule	मुख्य काम
12	Connection Metrics	Usage और performance metrics

अब प्रत्येक को विस्तार से देखते हैं।

3. Submodule 01 — Connection Registry

यह सभी registered PostgreSQL server connections का master registry होगा।

इसके काम:

- नया connection register करना।
- Existing connection edit करना।
- Connection delete करना।
- Connection duplicate करना।
- Connection को enable/disable करना।
- Project के साथ connection map करना।
- Connection details देखना।
- Connection profile export/import (बिना secrets के)।

उदाहरण:

Registered Connections

Main VPS
Connected

127.0.0.1:5432 · PostgreSQL

E-commerce VPS
Connected

db.example.internal:5432

IoT VPS
Disconnected

iot.example.internal:5432

एक Connection Profile में ये fields रहेंगी:

Field	Description
connection_id	UUID
connection_name	पहचानने योग्य नाम
host	Server hostname/IP
port	PostgreSQL port
username	Database user
credential_id	Encrypted credential reference
ssl_mode	TLS configuration
connection_timeout	Connection timeout
project_mapping	Associated projects
created_at	Creation timestamp
updated_at	Last update

महत्वपूर्ण: Password को इस registry में plaintext में नहीं रखना है। केवल `credential_id` रखना है।

4. Submodule 02 — Connection Configuration

इसका काम अलग-अलग PostgreSQL environments को configure करना है।

Configuration के प्रकार:

A. Local PostgreSQL

```
Host: 127.0.0.1
Port: 5432
Database: postgres
```

B. Remote PostgreSQL

```
Host: db.example.com
Port: 5432
Database: postgres
SSL: Required
```

C. Private Network PostgreSQL

```
Host: 10.0.0.10
Port: 5432
Network: Private
```

D. SSH Tunnel Connection

```
Local Port: 15432
Remote Host: 127.0.0.1
Remote Port: 5432
SSH Host: server.example.com
```

SSH tunnel के लिए dedicated tunnel component इस्तेमाल करेंगे। SQLx को tunnel के local endpoint से connect कराया जाएगा।

SSL Modes

PostgreSQL में TLS configuration बहुत जरूरी है।

Mode	उपयोग
Disable	केवल trusted local environment में
Require	TLS encryption अनिवार्य
Verify CA	Certificate authority verify
Verify Full	CA और hostname दोनों verify

मेरी recommendation: Remote production connections में `Verify Full` default रखो। Self-signed certificates के लिए trusted CA configuration उपलब्ध कराओ, certificate verification बंद करना सामान्य समाधान नहीं होना चाहिए।

5. Submodule 03 — Credential Manager

यह सबसे sensitive component होगा।

इसके अंदर:

- Password encryption
- Credential storage
- Credential rotation

- Password update
- Secret retrieval
- Secret deletion
- Access auditing
- Encryption key management

Architecture:

Encrypted Credential Storage

Credential Manager
Authorization check

Encryption / Decryption
AES-256-GCM

Connection Manager
Temporary access to secret

Recommended security design:

- AES-256-GCM authenticated encryption।
- Encryption key VPS पर database से अलग रखी जाएगी।
- Key rotation का support।
- Password logs और API responses में कभी नहीं दिखेगा।
- Secrets केवल authorized backend component को मिलेंगे।
- Credentials को memory में अनावश्यक रूप से retain नहीं करेंगे।

एक अतिरिक्त निर्णय: PostgreSQL application user को superuser नहीं बनाना है। हर project के लिए न्यूनतम आवश्यक privileges वाला account इस्तेमाल करना बेहतर है।

6. Submodule 04 — Connection Tester

जब तुम नया PostgreSQL connection जोड़ोगे, तो यह component उसकी जाँच करेगा।

Testing pipeline:

Host format validation

DNS resolution

TCP connectivity test

TLS certificate verification

PostgreSQL authentication

Database access verification

Permission verification

Connection latency measurement

उदाहरण के लिए:

Connection Test Result

DNS Resolution *Passed*

TCP Connection *Passed*

TLS Verification *Passed*

Authentication *Passed*

Database Access *Passed*

Connection test successful

ध्यान देना: Test के लिए एक अस्थायी connection बनाया जाएगा और test समाप्त होने पर बंद कर दिया जाएगा। Test के नाम पर स्थायी connection pool नहीं बनाना है।

7. Submodule 05 — Connection Pool Manager

यह सबसे महत्वपूर्ण performance component है।

हर SQL request के लिए नया PostgreSQL connection बनाना महँगा होता है। इसलिए SQLx connection pool का इस्तेमाल करेंगे।

उदाहरण:

```
Connection Pool
       |
       ├── Connection 01
       ├── Connection 02
       ├── Connection 03
       ├── Connection 04
       └── Connection 05
```

जब कोई request आएगी:

1. Pool से उपलब्ध connection लिया जाएगा।
2. SQL query execute होगी।
3. Connection वापस pool में जाएगा।
4. अगली request उसी connection का reuse कर सकती है।

Recommended initial configuration

ये शुरुआती configuration values हैं, final performance limits नहीं। इन्हें benchmark के बाद tune करना होगा।

Setting	Initial value
Max connections / pool	5
Min connections / pool	0
Acquire timeout	5 seconds
Connect timeout	5 seconds
Idle timeout	5 minutes
Max lifetime	30 minutes

सबसे जरूरी architectural निर्णय

तुम्हारे पास 20 Databases हों तो 20 pools बनाकर हर एक में 5 connections reserve करना सही नहीं होगा।

इसके बजाय:

- Lazy pool creation।
- Bounded total pool count।
- Per-server और global connection budgets।
- Idle pool eviction।
- Active operation के दौरान pool eviction नहीं।
- जरूरत के अनुसार pool पुनः create करना।

उदाहरण:

```
Global Connection Budget: 30

Main VPS
  ├── ecommerce_db: 5 max
```

```
  ├── converter_db: 5 max
  └── imgforge_db: 5 max

Remote VPS
  ├── saas_db: 5 max
  └── vending_db: 5 max
```

यहाँ 30 global budget है, लेकिन वास्तविक concurrent connections इससे कम भी रह सकते हैं। हर server की अपनी PostgreSQL connection limit भी ध्यान में रखनी होगी।

8. Submodule 06 — Pool Lifecycle Manager

Pool बनाना ही काफी नहीं है। उसे कब बंद करना है, कब दोबारा बनाना है और कब remove करना है, यह भी तय करना होगा।

Operations:

Operation	काम
Create Pool	नया database pool
Acquire	Pool से connection लेना
Release	Connection वापस करना
Close Pool	Pool बंद करना
Evict Pool	Idle pool हटाना
Recreate Pool	Configuration change के बाद नया pool
Drain Pool	नए requests रोककर active work पूरा करना

उदाहरण:

```
Pool Created
     ↓
Active
     ↓
Idle
     ↓
Idle Timeout
     ↓
Eviction
```

यदि कोई connection active transaction में है, तो उसे अचानक terminate नहीं करना है।

Password बदलने या server configuration update होने पर पुराने pool को drain करके नए configuration से pool तैयार करना होगा।

9. Submodule 07 — Health Monitor

यह लगातार सभी connections की स्थिति पर नजर रखेगा।

Health checks:

- Server reachable है या नहीं।
- PostgreSQL responding है या नहीं।
- Authentication सही है या नहीं।
- Pool acquire हो रहा है या नहीं।
- कितने connections इस्तेमाल हो रहे हैं।
- कितने idle हैं।
- कितनी requests wait कर रही हैं।
- Last successful health check कब हुआ।

एक बात: हर कुछ सेकंड में हर database को query करके health check करना भी unnecessary load पैदा कर सकता है। इसलिए configurable intervals और lightweight checks रखेंगे।

10. Submodule 08 — Reconnection Manager

मान लो तुम्हारा VPS restart हो गया या इंटरनेट disconnect हो गया।

ऐसी स्थिति में:

```
Connection Lost
      ↓
Failure Detected
      ↓
Check Configuration
      ↓
Retry Delay
      ↓
Reconnect Attempt
      ↓
Success / Failed
```

Recommended retry strategy:

- Exponential backoff

- Random jitter
- Maximum retry limit
- Configurable retry interval
- Manual reconnect
- Automatic reconnect

उदाहरण:

Attempt	Retry delay
1	1 second
2	2 seconds
3	4 seconds
4	8 seconds
5	16 seconds

यह केवल retry schedule का उदाहरण है। वास्तविक retry में jitter और maximum delay भी लागू होंगे।

महत्वपूर्ण: किसी failed SQL transaction को blindly दोबारा execute नहीं करना है। Connection टूटने पर यह पता नहीं भी चल सकता कि transaction commit हुई थी या नहीं। ऐसे मामलों में operation result verify करना जरूरी होगा।

11. Submodule 09 — Connection State Manager

हर connection की स्थिति स्पष्ट होनी चाहिए।

हम ये states रखेंगे:

State	मतलब
`Registered`	Profile सेव है
`Connecting`	Connection बन रहा है
`Connected`	Connection सफल
`Disconnected`	अभी connected नहीं
`Degraded`	Connection है, लेकिन समस्या है
`Reconnecting`	दोबारा connect हो रहा है

State	मतलब
`Failed`	Connection असफल
`Disabled`	Admin ने बंद किया
`Closing`	Pool बंद हो रहा है

एक महत्वपूर्ण अंतर: `Connected` का मतलब यह नहीं कि हर database में हर समय active TCP connection मौजूद रहेगा। Lazy pool में pool उपलब्ध हो सकता है, भले उसमें अभी कोई physical connection न हो।

12. Submodule 10 — Network Manager

यह remote PostgreSQL servers से network connectivity संभालेगा।

इसमें शामिल होंगे:

- DNS Resolver
- TCP Connector
- TLS Handshake
- Certificate Verification
- SSH Tunnel
- Connection Timeout
- Network Failure Detection
- Proxy Support (यदि भविष्य में जरूरी हो)

विशेष सुरक्षा: क्योंकि तुम्हारा Manager किसी भी external database से जुड़ सकता है, इसलिए connection creation को SSRF और network abuse से सुरक्षित रखना जरूरी है।

उदाहरण के लिए:

- Arbitrary metadata endpoints तक पहुँच रोकना।
- Private IP access केवल authorized configuration में देना।
- DNS resolution के बाद destination IP validate करना।
- DNS rebinding से सुरक्षा।
- Connection destinations का audit रखना।

तुम्हारे अपने VPS और private network databases के लिए private IP connections की अनुमति रहेगी, लेकिन unrestricted network access नहीं।

13. Submodule 11 — Connection Isolation

यह सुनिश्चित करेगा कि किसी प्रोजेक्ट का connection दूसरे प्रोजेक्ट के द्वारा इस्तेमाल न हो सके।

उदाहरण:

```
Project A
  └── ecommerce_connection
        └── ecommerce_db

Project B
  └── converter_connection
        └── converter_db
```

सिर्फ UUID जान लेने से दूसरे project का connection access नहीं होना चाहिए।

हर request में:

1. User authentication।
2. Project permission।
3. Connection permission।
4. Database mapping verification।
5. Operation authorization।

इन सभी की जाँच होगी।

14. Submodule 12 — Connection Metrics

यह module performance information उपलब्ध कराएगा।

Metric	मतलब
Total Profiles	Registered connections
Active Pools	वर्तमान pools
Active Connections	इस्तेमाल हो रहे connections
Idle Connections	खाली connections
Failed Connections	Failed connection attempts
Average Latency	औसत connection latency

Metric	मतलब
Pool Wait Time	Connection के लिए प्रतीक्षा
Reconnect Count	Reconnection attempts

इन metrics को simple text और tables में दिखाना पर्याप्त होगा। तुम्हें charts बनाने की आवश्यकता नहीं है।

---

15. Connection Manager का Complete Workflow

अब इन सभी submodules को एक साथ जोड़ते हैं।

New Connection Workflow

Slint से Add Connection request।

API Gateway request validation।

Connection Registry profile तैयार करेगा।

Credential Manager encrypted password सुरक्षित करेगा।

Connection Tester server की जाँच करेगा।

Project और database mapping verify होगी।

Connection profile registry में save होगा।

Database चुनने पर Pool Manager lazy pool बनाएगा।

Health Monitor और State Manager उसकी स्थिति संभालेंगे।

Connection Ready

Runtime workflow

```
Slint Request
      |
      v
Authentication
      |
      v
Project Permission
      |
```

```
      v
Connection Registry
      |
      v
Pool Manager
      |
      v
SQLx Connection Pool
      |
      v
PostgreSQL
      |
      v
Result / Error
      |
      v
Slint UI
```

16. Connection Manager का Rust Folder Structure

मैं इस module को बाकी backend से स्वतंत्र रखना चाहूँगा।

```
src/
└── connections/
    │
    ├── mod.rs
    ├── controller.rs
    ├── service.rs
    ├── repository.rs
    ├── models.rs
    ├── dto.rs
    │
    ├── registry/
    │   ├── mod.rs
    │   ├── create.rs
    │   ├── update.rs
    │   └── delete.rs
    │
    ├── config/
    │   ├── mod.rs
    │   ├── postgres.rs
    │   ├── ssl.rs
    │   └── validation.rs
    │
    ├── credentials/
    │   ├── mod.rs
    │   ├── encryption.rs
    │   ├── decryption.rs
    │   ├── vault.rs
```

```
│   └── rotation.rs
│
├── testing/
│   ├── mod.rs
│   ├── dns.rs
│   ├── tcp.rs
│   ├── tls.rs
│   ├── authentication.rs
│   └── permissions.rs
│
├── pool/
│   ├── mod.rs
│   ├── manager.rs
│   ├── factory.rs
│   ├── lifecycle.rs
│   ├── eviction.rs
│   └── limits.rs
│
├── health/
│   ├── mod.rs
│   ├── monitor.rs
│   └── checker.rs
│
├── reconnect/
│   ├── mod.rs
│   ├── retry.rs
│   └── backoff.rs
│
├── state/
│   ├── mod.rs
│   └── machine.rs
│
├── network/
│   ├── mod.rs
│   ├── resolver.rs
│   ├── connector.rs
│   ├── tunnel.rs
│   └── security.rs
│
├── isolation/
│   ├── mod.rs
│   └── access.rs
│
├── metrics/
│   ├── mod.rs
│   └── collector.rs
│
└── tests/
    ├── registry_tests.rs
    ├── pool_tests.rs
    ├── security_tests.rs
    ├── reconnect_tests.rs
    └── integration_tests.rs
```

यह logical structure है। शुरुआत में हर छोटी file अलग बनाना जरूरी नहीं है। एक file में 300–500 lines की guideline रख सकते हो, लेकिन केवल line count के कारण code को कृत्रिम रूप से विभाजित नहीं करना है।

17. Management Database में Connection Tables

इस module के लिए मुख्य tables:

Table	Purpose
`connections`	Server connection profiles
`connection_credentials`	Encrypted credentials
`connection_ssl_configs`	TLS configurations
`connection_projects`	Project mappings
`connection_databases`	Registered database targets
`connection_health_logs`	Health history
`connection_events`	Connection state events
`connection_metrics`	Performance metrics
`connection_access`	Connection permissions

एक और निर्णय: Password, private key और encryption key को अलग-अलग secrets के रूप में संभालना है। SSH private key और PostgreSQL password को एक ही field में नहीं रखना है।

18. Connection Manager API Blueprint

Endpoint	Method	काम
`/connections`	GET	सभी profiles
`/connections`	POST	नया profile
`/connections/{id}`	GET	Profile details
`/connections/{id}`	PATCH	Configuration update

Endpoint	Method	काम
`/connections/{id}`	DELETE	Profile remove
`/connections/{id}/test`	POST	Test connection
`/connections/{id}/connect`	POST	Pool initialize
`/connections/{id}/disconnect`	POST	Pool drain/close
`/connections/{id}/health`	GET	Health status
`/connections/{id}/metrics`	GET	Performance data
`/connections/{id}/reconnect`	POST	Manual reconnect
`/connections/{id}/databases`	GET	Database targets

यहाँ `/connect` का अर्थ persistent pool initialize करना होगा, किसी एक TCP connection को हमेशा खुला रखना नहीं।

19. Development Priority

मैं इस module को चार चरणों में बनाऊँगा।

Phase 1 — Basic Connection Engine

पहले

- Connection Registry
- Configuration
- Credentials
- Connection Tester
- PostgreSQL connectivity

Phase 2 — Pool Engine

- SQLx Pool Manager
- Pool Lifecycle
- Global connection limits
- Pool eviction
- Connection isolation

Phase 3 — Reliability

- Health Monitor
- State Manager
- Reconnection
- Network Manager
- SSH Tunnel

Phase 4 — Production Optimization

- Metrics
- Credential rotation
- Failure recovery
- Load testing
- Security testing

---

20. मेरे अनुसार अभी तीन निर्णय Final करने जरूरी हैं

ताकि अगले modules में कोई architectural conflict न आए।

1. PostgreSQL Connections कहाँ से आएँगे?

- सिर्फ एक VPS के अंदर मौजूद PostgreSQL
- सिर्फ Remote VPS और External Servers
- दोनों — Local और Remote PostgreSQL

---

2. Connection Pool Strategy क्या होगी?

- हर Database का अलग Pool
- एक Shared Pool
- Hybrid — Server-level limits + Database-level lazy pools

---

3. Remote Connection कैसे होगी?

- Direct TCP + TLS
- SSH Tunnel
- दोनों — Direct TLS और SSH Tunnel

मेरी recommendation: तीनों में `Both / Hybrid / Both` सबसे उपयोगी combination है। इससे तुम्हारा Database Manager भविष्य में केवल अपने VPS तक सीमित नहीं रहेगा और अलग-अलग प्रोजेक्ट्स के databases को भी एक ही जगह से नियंत्रित कर सकेगा।

You:

Connection Manager के final architectural decisions:
PostgreSQL scope: both
Pool strategy: hybrid
Network connectivity: both
इनके आधार पर Connection Manager का detailed final specification तैयार करो।

ChatGPT:

Connection Manager — Final Architectural Specification

यह specification आपके final decisions पर आधारित है:

- **PostgreSQL scope:** Local और remote दोनों
- **Pool strategy:** Hybrid — server-level resource budgets + database-specific lazy pools
- **Network connectivity:** Direct TCP/TLS और SSH tunnel दोनों

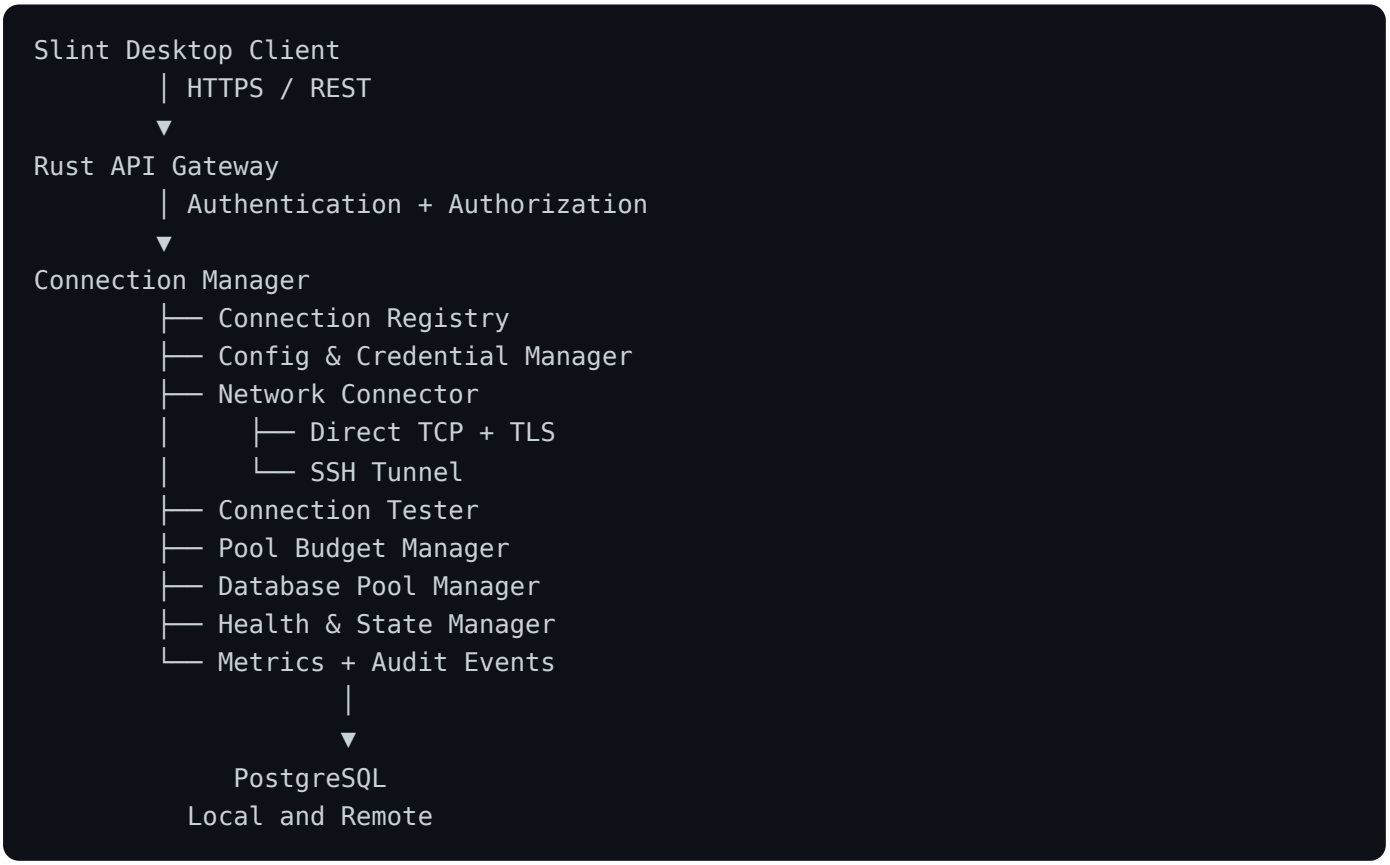
1. उद्देश्य और सीमाएँ

Connection Manager, VPS पर चलने वाले Rust backend का वह subsystem होगा जो PostgreSQL servers और उनके databases के लिए सुरक्षित connections बनाता, जांचता, pool करता, monitor करता और बंद करता है।

मुख्य सीमा: Slint desktop client सीधे PostgreSQL से connect नहीं करेगा। वह authenticated HTTPS API के जरिए Rust backend को request भेजेगा। Backend ही connection credentials और database access संभालेगा।

Connection Manager database schema या records का business logic नहीं संभालेगा; वह अन्य modules को नियंत्रित database connections उपलब्ध कराएगा।

2. Architecture



Design pattern: Rust modular monolith. Connection Manager को शुरुआत में अलग microservice न बनाएं; इसके modules स्पष्ट interfaces के साथ उसी backend process में रहें।

3. Core design decisions

3.1 Server profile और database target अलग होंगे

एक PostgreSQL server profile में host, port, TLS और authentication configuration होगी। उस server पर मौजूद हर database एक अलग target होगा।

- **Server profile:** PostgreSQL instance तक पहुंचने का तरीका।
- **Database target:** उस instance पर एक विशिष्ट database।
- **Pool:** किसी database target के लिए SQLx connection pool।

एक server profile से कई databases discover हो सकते हैं, लेकिन हर database का pool अलग और **lazy-created** होगा—pool तभी बनेगा जब उस database पर वास्तविक operation की जरूरत हो।

3.2 Hybrid pool strategy

Hybrid का अर्थ:

1. **Server-level budget:** एक PostgreSQL server पर कुल कितने pooled connections इस्तेमाल हो सकते हैं, इसकी सीमा।
2. **Database-specific pools:** हर database target का अपना SQLx pool।
3. **Global budget:** सभी server profiles के pools का संयुक्त connection budget।
4. **Lazy creation और idle eviction:** उपयोग न होने वाले pools को अनावश्यक रूप से बनाए न रखें।

SQLx का `max_connections` प्रत्येक pool पर लागू होता है; इसलिए केवल per-pool limit पर्याप्त नहीं है। Pool Budget Manager को global और server-level limits लागू करनी होंगी।

3.3 Local और remote PostgreSQL

- **Local:** उसी VPS या backend के अधिकृत private network में चल रहा PostgreSQL।
- **Remote:** अलग host या network पर चल रहा PostgreSQL।
- दोनों के लिए समान profile और pool lifecycle रहेगा; network route अलग हो सकता है।
- Local connection के लिए भी authorization और TLS policy लागू रहेगी। “Local” को स्वत: trusted न मानें।

3.4 Direct TCP/TLS और SSH tunnel

- **Direct TCP/TLS:** सामान्य प्राथमिक route।

- **SSH tunnel:** जब network policy, firewall या deployment topology इसकी मांग करे।
- प्रत्येक profile में चुना हुआ route स्पष्ट दर्ज होगा; connection attempt के बीच चुपचाप route बदलना नहीं होगा।
- SSH tunnel process/forwarding lifecycle को PostgreSQL pool lifecycle से जोड़ा जाएगा, ताकि pool बंद होने पर tunnel भी नियंत्रित तरीके से बंद हो सके।

4. Modules और जिम्मेदारियां

Module	जिम्मेदारी
`registry`	Server profiles और database targets का lifecycle
`config`	Host, port, TLS mode, timeout और route settings की validation
`credentials`	Credentials का encrypted storage, retrieval और rotation
`network`	DNS, TCP, TLS और SSH tunnel setup/teardown
`testing`	बिना persistent pool बनाए connection test
`pool_budget`	Global और per-server connection limits
`pool`	Database-specific SQLx pools बनाना और acquire/release करना
`lifecycle`	Pool create, drain, close, evict और recreate
`health`	Connectivity और pool usability checks
`reconnect`	Backoff, retry limits और manual reconnect
`state`	Connection और pool states का authoritative tracking
`isolation`	User, project और database target permissions की जांच
`metrics`	Pool usage, latency, failures और reconnect metrics
`audit`	Sensitive operations और configuration changes का audit trail

5. Connection profile और configuration

Server profile में कम-से-कम ये fields हों:

```
connection_id
display_name
scope                  // local | remote
host
```

```
port
username
credential_id
network_mode           // direct_tls | ssh_tunnel
tls_mode
connect_timeout
acquire_timeout
idle_timeout
max_lifetime
enabled
created_at
updated_at
```

Database target में:

```
database_target_id
connection_id
database_name
pool_policy_id
enabled
last_discovered_at
```

`database_name` और अन्य SQL identifiers को सामान्य bind parameters की तरह treat न करें। उन्हें validated identifier handling के जरिए इस्तेमाल करें।

6. Credentials और TLS

- Credentials को plaintext में database, logs या API responses में न रखें।
- Encryption key को application database से अलग secret source में रखें।
- Key rotation और credential rotation का समर्थित workflow हो।
- Passwords, tokens, private keys और connection strings logs में redact हों।
- Remote direct connections के लिए production default **TLS certificate verification** हो; `verify-full` उपयुक्त default है।
- TLS verification disable करना सामान्य production configuration का हिस्सा न हो।
- SSH private keys को भी secret material की तरह संभालें; उनकी permissions और rotation policy रखें।

7. Connection test workflow

`Test Connection` temporary connection इस्तेमाल करेगा—यह persistent pool नहीं बनाएगा।

1. Request authentication और project/connection permission check।
2. Host, port और route configuration validation।

3. DNS resolution और allowed destination validation।
4. TCP connection या SSH tunnel establishment।
5. TLS handshake और certificate verification।
6. PostgreSQL authentication।
7. चुने गए database तक access और आवश्यक permissions check।
8. Latency और diagnostic result दर्ज करना।
9. Temporary connection और tunnel को साफ तरीके से बंद करना।

Test result में success/failure, stage, elapsed time और sanitized error message लौटे। Credentials या raw connection string response में न आए।

8. Pool lifecycle और limits

Pool lifecycle:

```
Uninitialized
   → Creating
   → Ready
   → Draining
   → Closed
```

Failure या network loss पर pool `Unhealthy` या `Failed` state में जा सकता है।

Pool budget policy

- Global, server-level और database-level limits अलग configuration scopes होंगे।
- Pool create करने से पहले budget reservation जरूरी है।
- Budget न मिलने पर request controlled तरीके से reject या queue हो; unlimited pools न बनें।
- Idle eviction केवल ऐसे pools पर हो जिनमें active operations न हों।
- Pool close करते समय नए acquisitions रोकें, active operations को निर्धारित grace period दें और फिर close करें।
- Pool limits को production में workload और PostgreSQL capacity के आधार पर benchmark करके tune करें; कोई fixed limit सभी deployments के लिए सही नहीं होगी।

9. Health, reconnect और state

Health checks

Health monitor pool usability, acquire latency, failed connection attempts और last successful check देखेगा। Health check का परिणाम वास्तविक user query की सफलता की गारंटी नहीं होगा।

Reconnect policy

- Exponential backoff और jitter।
- Configurable retry ceiling और cooldown।
- Manual reconnect action।
- Reconnect loop के दौरान concurrent reconnect attempts को single-flight तरीके से रोकें।
- Network error के बाद किसी SQL transaction को blindly retry न करें—यह पता न हो सकता है कि transaction server पर commit हुआ था या नहीं।

State model

```
Registered
Connecting
Connected
Disconnected
Degraded
Reconnecting
Failed
Disabled
Closing
```

`Connected` का अर्थ है कि target का pool उपयोग के लिए उपलब्ध है; यह जरूरी नहीं कि उस क्षण कोई physical connection active हो।

10. Network security और isolation

- हर request पर user → project → connection → database target permission verify करें।
- DNS और destination validation के जरिए SSRF तथा DNS-rebinding जोखिम घटाएं।
- Private IP destinations केवल स्पष्ट रूप से authorized configuration के तहत स्वीकार हों।
- SSH tunnel configuration में host key verification हो; “accept any host key” production default न हो।
- PostgreSQL port को public internet पर expose न करें जब तक deployment में इसकी स्पष्ट आवश्यकता और सुरक्षा व्यवस्था न हो।

- Connection Manager को arbitrary shell commands चलाने की अनुमति न दें। PostgreSQL service start/stop अलग privileged subsystem का काम होगा।

11. API surface

```
POST   /connections
GET    /connections
GET    /connections/{id}
PATCH  /connections/{id}
DELETE /connections/{id}

POST   /connections/{id}/test
POST   /connections/{id}/connect
POST   /connections/{id}/disconnect
POST   /connections/{id}/reconnect

GET    /connections/{id}/health
GET    /connections/{id}/metrics
GET    /connections/{id}/databases
```

API credentials को केवल create/update request में स्वीकार करे; list/detail responses में secrets वापस न लौटाए। सभी endpoints project-level authorization और audit policy के अधीन हों।

12. Persistence model

प्रस्तावित tables:

```
connections
connection_credentials
connection_ssl_configs
connection_ssh_configs
connection_projects
connection_databases
connection_pool_policies
connection_health_logs
connection_events
connection_access
```

Pool objects runtime resources हैं; उन्हें database में serialize करके persist न करें। Database में केवल उनकी policy और आवश्यक operational metadata रखें।

13. Metrics और audit

Metrics में:

- Registered server profiles और database targets
- Active pools तथा active/idle connections
- Pool acquire wait time और connection latency
- Connection/test failures
- Reconnect attempts और outcomes
- Pool eviction और budget rejection counts

Audit में profile create/update/delete, credential rotation, test/connect/disconnect, permission changes और reconnect actions दर्ज हों। Audit logs में secrets न हों।

14. Suggested source layout

```
src/connections/
├── mod.rs
├── registry/
├── config/
├── credentials/
├── network/
│   ├── direct_tls.rs
│   └── ssh_tunnel.rs
├── testing/
├── pool_budget/
├── pool/
├── lifecycle/
├── health/
├── reconnect/
├── state/
├── isolation/
├── metrics/
└── audit/
```

15. Implementation sequence

1. Registry, config validation और project authorization।
2. Direct TCP/TLS connection test।
3. Database discovery और database-specific lazy SQLx pools।
4. Global/server pool budget manager और lifecycle handling।
5. SSH tunnel route और tunnel cleanup।
6. Health checks, reconnect policy, metrics और audit।

7. Failure testing: invalid credentials, DNS/TLS failure, network interruption, exhausted budgets, idle eviction और shutdown के दौरान active queries।

Final architecture: एक Rust Connection Manager, जिसमें local और remote PostgreSQL profiles समान control plane से manage होंगे; direct TLS और SSH tunnel दोनों supported होंगे; और हर database का lazy SQLx pool global तथा per-server budgets के भीतर चलेगा।

You:

Part 2 — File Storage & Upload System
Images, videos, documents, upload और download

ChatGPT:

Part 2 — File Storage & Upload System

इस module का उद्देश्य आपके VPS-hosted platform पर projects के लिए **images, videos और documents** को सुरक्षित रूप से upload, store, manage और download करना है। यह Part 1 के Rust backend और Connection Manager से अलग रहेगा: PostgreSQL में file metadata और permissions रहेंगी; actual file bytes VPS disk या compatible object storage में।

1. Architecture

```
Slint Desktop Client / Future Web or Mobile Apps
                    | HTTPS
                    ▼
              Rust API Gateway
                    | Auth + Project Permissions
                    ▼
             File Storage Module
       ├── Upload Session Manager
       ├── Validation & File Inspection
       ├── Storage Adapter
       ├── Metadata Manager
       ├── Download / Access Manager
       ├── Lifecycle & Cleanup
       └── Audit + Metrics
                    |
          ┌─────────┴─────────┐
          ▼                   ▼
```

```
   PostgreSQL Metadata    File Storage
                         VPS Disk / S3-compatible
```

मुख्य सिद्धांत: PostgreSQL में file content को बड़े binary blobs के रूप में नहीं रखा जाएगा। Database में metadata, ownership, storage key, status और access rules रहेंगे; file bytes storage layer में।

2. Storage backend

Phase 1 — VPS disk

शुरुआत में VPS disk पर private storage directory इस्तेमाल की जा सकती है। Files को user-provided filename के आधार पर सीधे path में न रखें; server-generated opaque storage key इस्तेमाल करें।

उदाहरण:

```
/storage/
  └── <opaque-storage-key>
```

यह केवल conceptual layout है—वास्तविक path deployment configuration से आएगा।

Future — Object storage

Storage Adapter interface ऐसा रखें कि बाद में S3-compatible object storage जोड़ा जा सके, बिना upload/download API को दोबारा design किए।

```
StorageAdapter
├── put()
├── get()
├── delete()
├── exists()
└── get_metadata()
```

VPS disk और object storage को एक ही abstraction के पीछे रखें; storage key और provider-specific path को client से छिपाएं।

3. File metadata model

प्रस्तावित `files` table:

Field	उद्देश्य
`file_id`	Unique file identifier

Field	उद्देश्य
`project_id`	Project ownership
`owner_user_id`	Upload करने वाला user
`original_name`	User द्वारा दिया गया filename
`storage_key`	Internal opaque storage reference
`media_type`	Validated MIME type
`extension`	Normalized extension
`size_bytes`	Stored file size
`checksum`	Integrity verification के लिए hash
`storage_provider`	`local` या configured provider
`status`	Upload/scan/ready/deleted state
`created_at`	Upload time
`deleted_at`	Soft-delete timestamp, यदि लागू हो

अलग tables रखें:

- `upload_sessions`
- `file_permissions`
- `file_versions` — यदि versioning चाहिए
- `file_events` — audit trail
- `file_storage_usage` — project/user quotas के लिए

4. Upload workflow

1. Client authenticated API को upload request भेजता है।
2. Backend user, project permission, quota और file-size policy जांचता है।
3. Upload session बनती है और temporary storage key allocate होती है।
4. Client file को chunks या stream के रूप में upload करता है।
5. Backend size limit, MIME/signature और checksum validate करता है।
6. जरूरत के अनुसार malware scanning या quarantine step चलता है।

7. File temporary area से final storage location में commit होती है।
8. PostgreSQL metadata `ready` status में update होती है।
9. Client को `file_id` और sanitized metadata मिलता है।

अधूरी uploads को session expiry और cleanup worker द्वारा हटाया जाएगा। Final metadata तभी प्रकाशित हो जब storage write सफल हो।

5. Upload modes

- **Small files:** सीधे authenticated multipart upload।
- **Large files:** resumable/chunked upload, ताकि network interruption पर पूरा upload दोबारा न करना पड़े।
- **Future direct-to-object-storage:** यदि object storage अपनाया जाए, तो short-lived signed upload URLs का विकल्प जोड़ सकते हैं।

Chunk size, max file size और concurrency limits deployment और workload के अनुसार configurable हों; बिना benchmark के hardcoded performance promises न रखें।

6. Download workflow

1. Client `file_id` के लिए download request करता है।
2. Backend user/project/file permissions जांचता है।
3. File `ready` state में है या नहीं, verify करता है।
4. Backend controlled stream या short-lived signed URL देता है।
5. Download event और relevant metrics दर्ज होते हैं।

Private files के लिए public, permanent URLs न बनाएं। Download response में safe `Content-Disposition` और validated content type दें। Untrusted files को browser में inline render करने के बजाय attachment के रूप में भेजना safer default है।

7. File validation और security

- Filename को path या storage key की तरह इस्तेमाल न करें।
- File extension, declared MIME type और file signature को अलग-अलग validate करें।
- File size, per-project storage quota और concurrent upload limits लागू करें।
- Uploads को non-executable storage location में रखें।
- Archive extraction जैसे features हों तो path traversal और decompression-bomb risks अलग से handle करें।

- Private file access हर बार authorize हो; केवल obscure `file_id` access control नहीं है।
- Temporary upload tokens short-lived और narrowly scoped हों।
- Logs में file contents, secrets या signed URLs न लिखें।
- Backup और deletion policy metadata तथा actual file bytes दोनों पर लागू हो।

8. File lifecycle

```
Created
  → Uploading
  → Validating
  → Ready
  → Deleted
```

Failure पर session `Failed` हो सकती है। `Deleted` file को configured retention period के बाद physical storage से हटाया जा सकता है। Cleanup job metadata और storage state के mismatch को भी detect करे।

9. API surface

```
POST   /files/upload-sessions
POST   /files/upload-sessions/{id}/complete
GET    /files/upload-sessions/{id}
DELETE /files/upload-sessions/{id}

GET    /files
GET    /files/{file_id}
GET    /files/{file_id}/download
DELETE /files/{file_id}
```

Large/resumable upload के लिए chunk endpoints और upload-session token format अलग से define होंगे। API में client को filesystem path या raw storage key न लौटाएं।

10. Rust module layout

```
src/files/
├── mod.rs
├── api/
├── metadata/
├── upload/
│   ├── sessions.rs
│   ├── multipart.rs
│   └── resumable.rs
├── validation/
```

```
├── storage/
│   ├── adapter.rs
│   ├── local_disk.rs
│   └── object_store.rs
├── download/
├── permissions/
├── quotas/
├── lifecycle/
├── cleanup/
├── audit/
└── metrics/
```

11. Implementation phases

1. File metadata, project permissions और local-disk storage adapter।
2. Small-file upload, validation और controlled download।
3. Upload sessions, resumable/chunked uploads और cleanup।
4. Quotas, audit, metrics और backup/restore tests।
5. Malware scanning/quarantine और object-storage adapter, आवश्यकता के अनुसार।

Final design: Rust backend file access का एकमात्र control point रहेगा। PostgreSQL metadata और permissions रखेगा; file bytes private storage में रहेंगी। Upload/download project-scoped authorization, quotas, validation और lifecycle cleanup से नियंत्रित होंगे।

You:

Part 3 — Security & Access Control
Authentication, project isolation और file permissions

ChatGPT:

Part 3 — Security & Access Control

इस module का काम यह तय करना है कि **कौन user**, **किस project में**, **कौन-सा action** कर सकता है—और हर API request पर उस अधिकार को enforce करना। Authentication, project isolation और file permissions को एक shared authorization layer में लागू करें, ताकि अलग-अलग modules अपने-अपने नियम न बनाएं।

1. Security architecture



Rule: Authentication बताता है कि user कौन है; authorization बताता है कि वह requested resource पर action कर सकता है या नहीं। Valid login अपने-आप project या file access नहीं देता।

2. Authentication और session management

User identity

हर user का stable `user_id` होगा। Login flow में credentials verify होने के बाद backend session या token जारी करेगा।

Session policy

- Access token short-lived रखें।
- Refresh token को rotate करें और reuse detection रखें।
- Logout, password reset या account disable होने पर active sessions revoke करने का रास्ता हो।
- Passwords को Argon2id जैसे password-hashing algorithm से hash करें; plaintext password कभी store न करें।
- Login failures पर rate limiting और suspicious activity logging रखें।
- Password reset tokens random, short-lived, single-use और hashed-at-rest हों।

API request flow

1. Token/session validate करें।
2. User active है या disabled, जांचें।
3. Requested project और resource का authorization करें।
4. Permission मिलने पर ही operation execute करें।
5. Sensitive actions का audit event लिखें।

3. Project isolation

हर project एक security boundary होगा। User कई projects का member हो सकता है, लेकिन एक project की membership दूसरे project का access नहीं देती।

Core entities

```
users
projects
project_members
roles
permissions
role_permissions
project_member_roles
sessions
refresh_tokens
```

Isolation rules

- हर project-owned resource में `project_id` रखें—connections, database targets, files, upload sessions और audit events सहित।
- Request में आए `project_id` पर भरोसा न करें; authenticated user की membership server-side verify करें।
- Resource lookup में project scope शामिल करें, ताकि दूसरे project का ID अनुमान लगाने पर data न मिले।
- Cross-project access केवल explicit, audited administrative permission से हो।
- Background jobs और cleanup workers भी project-scoped authorization context के साथ काम करें।

4. Role-based access control (RBAC)

शुरुआत में simple roles रखें; जरूरत बढ़ने पर granular permissions जोड़ें।

Role	सामान्य अधिकार
`Owner`	Project settings, members और resources का पूर्ण नियंत्रण
`Admin`	Members और project resources manage करना; ownership transfer नहीं
`Editor`	अनुमत resources create/update करना
`Viewer`	Read-only access
`Custom`	चुनी हुई permissions का संयोजन

Role names अकेले enforcement नहीं हैं। Backend को permission checks जैसे `file.read`, `file.delete`, `connection.manage` और `project.members.manage` पर निर्णय लेना चाहिए।

5. File permissions

File access को project membership के साथ file-specific permissions से नियंत्रित करें।

Permission set

```
file.read
file.upload
file.update_metadata
file.download
file.delete
file.share
```

Access decision

File request पर backend:

1. `file_id` resolve करे।
2. File का `project_id` और owner निकाले।
3. User की project membership और role जांचे।
4. File-specific ACL लागू हो तो उसे जांचे।
5. Requested action की अनुमति न हो तो access deny करे।

महत्वपूर्ण: File ID का कठिन-से-अनुमानित होना authorization का विकल्प नहीं है। Private files के लिए permanent public URLs न बनाएं। Signed download URL इस्तेमाल हो तो वह short-lived, specific file और सीमित operation के लिए हो।

File sharing

यदि future में project के बाहर file share करना हो, तो share token को अलग capability की तरह model करें:

- Random और unguessable token
- Expiry और revoke support
- Read-only और download scope
- Optional download limit
- Token का hash store करना
- Audit event

6. Authorization enforcement

एक central policy interface रखें, जिसे API handlers और background jobs call करें:

```
authorize(user, project, resource, action) → allow | deny
```

Policy evaluation में कम-से-कम user status, project membership, role permissions, resource ownership/ACL और resource state शामिल हों।

- Default policy **deny** हो।
- Permission check fail या policy service error होने पर operation fail closed हो।
- Client-supplied role, owner ID या permission claims को authoritative न मानें।
- Resource fetch और authorization के बीच race condition से बचने के लिए sensitive mutations में उचित transaction/locking strategy रखें।

7. Security events और audit

इन actions को audit करें:

- Login success/failure, logout और token revocation
- Project member/role changes
- File upload, download, share और delete
- Connection profile या credentials में बदलाव
- Permission denial और privileged actions

Audit record में actor, project, action, resource ID, timestamp और result रखें। Passwords, access tokens, refresh tokens, private keys और signed URLs log न करें।

8. API surface

```
POST   /auth/login
POST   /auth/refresh
POST   /auth/logout
POST   /auth/password-reset/request
POST   /auth/password-reset/confirm

GET    /projects/{project_id}/members
POST   /projects/{project_id}/members
PATCH  /projects/{project_id}/members/{user_id}
DELETE /projects/{project_id}/members/{user_id}

GET    /projects/{project_id}/permissions
GET    /files/{file_id}/permissions
PUT    /files/{file_id}/permissions
DELETE /files/{file_id}/permissions/{subject_id}
```

हर endpoint पर authentication और project/resource authorization अनिवार्य हो। Error responses दूसरे projects या files के अस्तित्व की अनावश्यक जानकारी न दें।

9. Suggested Rust module layout

```
src/security/
├── mod.rs
├── auth/
│   ├── login.rs
│   ├── tokens.rs
│   ├── sessions.rs
│   └── password_reset.rs
├── users/
├── projects/
│   ├── membership.rs
│   └── isolation.rs
├── roles/
├── permissions/
│   ├── policy.rs
│   └── enforcement.rs
├── files/
│   ├── acl.rs
│   └── sharing.rs
├── rate_limit/
├── audit/
└── security_events/
```

10. Implementation order

1. User identity, password hashing और session lifecycle।
2. Project membership और default-deny authorization middleware।
3. Resource-level project isolation; पहले connections और files पर enforce करें।
4. File ACL और secure download/share flows।
5. Audit, rate limiting, revocation और security tests।
6. Cross-project access, expired sessions, guessed IDs, revoked share links और concurrent permission changes के negative tests।

Final design: एक central Security & Access Control module authentication, project membership, RBAC और file ACL enforce करेगा। सभी resources project-scoped होंगे; access default-deny होगा; और file sharing केवल explicit, revocable, limited-scope permissions से होगा।

You:

Part 4 — Slint Desktop Manager
सरल UI, database explorer और file manager

ChatGPT:

Part 4 — Slint Desktop Manager

इस desktop app का काम VPS पर चल रहे Rust backend को एक सरल, सुरक्षित control panel देना है। UI में **database explorer और file manager** होंगे—लेकिन PostgreSQL credentials, raw storage paths और privileged server operations client पर नहीं होंगे।

1. Architecture

```
Slint Desktop App
   ├── Login / Project Selector
   ├── Database Explorer
   ├── File Manager
   └── Connection & Upload Status
            | HTTPS / REST
            ▼
```



Slint client केवल UI और API client होगा। Database connections, file authorization और storage operations backend पर होंगे। Client को PostgreSQL port या VPS filesystem तक direct access नहीं दिया जाएगा।

2. UI design principles

- Simple control panel: forms, selects, buttons, lists और status text।
- Charts और heavy visualizations की जरूरत नहीं।
- Main content area में selected page; sidebar में navigation।
- हर action के लिए loading, success और error state।
- Destructive actions—जैसे database target हटाना या file delete करना—confirmation के बाद।
- Credentials/password fields masked हों; secrets को logs या local config में न लिखें।

3. Main navigation

```
Home
├── Projects
├── Database Explorer
│   ├── Connections
│   ├── Databases
│   ├── Tables
│   └── Table Details
├── File Manager
│   ├── Files
│   ├── Uploads
│   └── File Details
├── Activity / Audit
└── Settings
```

4. Database Explorer

Connections list

हर connection row में:

- Display name
- Local/remote scope
- Direct TLS/SSH tunnel route
- Health/status
- Available actions: Test, Connect, Disconnect, Edit

Status text स्पष्ट हो: `Connected`, `Disconnected`, `Degraded`, `Failed` आदि। "Connected" का अर्थ backend pool उपलब्ध है—जरूरी नहीं कि उसी क्षण physical connection active हो।

Add/Edit Connection

Form fields:

- Connection name
- Host और port
- Username
- Password/credential input
- Database selection
- Network mode: Direct TLS / SSH tunnel
- TLS configuration

UI credential को केवल secure API request में भेजे। Backend validation और secret storage का जिम्मेदार रहेगा।

Database and table browsing

Database Explorer का flow:

```
Project
  → Connection
    → Database
      → Tables
        → Table Details
```

Table Details में शुरुआती scope:

- Columns और data types
- Primary key और indexes की जानकारी
- Read-only record preview
- Permission-aware refresh

Table data edit/update को अलग phase में रखें; शुरुआत में read-only browsing accidental data modification का जोखिम घटाता है। SQL console को default screen में न रखें—यदि बाद में जोड़ा जाए तो उसे अलग permission, query limits और audit के साथ implement करें।

5. File Manager

File list

हर file row में:

- Original filename
- Type/category
- Size
- Upload time
- Status
- Actions: Download, Rename metadata, Delete

List केवल authenticated user के authorized project files दिखाएगी। Internal storage key या filesystem path UI में न दिखाएं।

Upload flow

```
Select Project
  → Select Files
    → Upload
      → Progress / Status
        → Complete or Error
```

UI में upload progress, failed state और retry action दिखाएं। Large/resumable uploads में backend upload session का status poll या API response से update किया जा सकता है।

File preview

Images और documents के लिए preview केवल तब दें जब backend access authorization पास हो। Videos के लिए शुरुआती version में download और basic metadata पर्याप्त है; inline streaming/preview को बाद के phase में रखा जा सकता है।

6. Shared UI states

हर API-backed page में ये states हों:

- `Loading`
- `Empty`
- `Ready`
- `Error`
- `PermissionDenied`

Network failure पर user को स्पष्ट retry action मिले। UI stale data को नया data बताकर न दिखाए; refresh timestamp या status text रखें।

7. Slint module layout

```
desktop_manager/
├── src/
│   ├── main.rs
│   ├── app_state.rs
│   ├── api_client/
│   ├── auth/
│   ├── navigation/
│   ├── database_explorer/
│   ├── file_manager/
│   ├── uploads/
│   ├── components/
│   └── error_handling/
├── ui/
│   ├── app.slint
│   ├── login.slint
│   ├── shell.slint
│   ├── database_explorer.slint
│   ├── file_manager.slint
│   └── components/
└── tests/
```

8. Implementation phases

1. Login, project selector और shared API client।
2. Connections list, add/edit form और connection test।
3. Database/table explorer—पहले read-only।
4. File list, upload progress, download और delete confirmation।
5. Error states, permission-denied UI, audit/activity view और usability tests।

Final design: Slint app एक lightweight desktop control panel होगा—simple navigation, database explorer और file manager के साथ। सभी sensitive operations Rust backend के authenticated API से होंगे; UI में charts, direct database access या raw filesystem access नहीं होगा।

You:

Part 5 — VPS Deployment & Storage Infrastructure
Disk management, backups, HTTPS और deployment
Part 6 — Realtime, Performance & Scaling
WebSocket, upload speed, caching और बड़े वीडियो

ChatGPT:

नीचे Part 5 और Part 6 की final architecture दी है। इन्हें Parts 1–4 के साथ जोड़कर देखें: **VPS पर Rust backend**, PostgreSQL में metadata, private file storage और Slint desktop client—सभी access authenticated HTTPS API से।# Part 5 — VPS Deployment & Storage Infrastructure

1. Deployment architecture

```
Slint Desktop Client
        | HTTPS
        ▼
Reverse Proxy (Caddy / Nginx)
        |
        ▼
Rust Backend (Axum)
   ├── Security & Access Control
   ├── Connection Manager
   └── File Storage Module
        |
   ├── PostgreSQL — metadata and control data
   └── Private File Storage — uploaded bytes
```

Deployment principle: Public traffic केवल HTTPS reverse proxy से आए। PostgreSQL और private file-storage directories public internet से सीधे accessible न हों।

2. VPS disk management

Storage को logical रूप से अलग रखें:

```
VPS
├── Application
│   ├── Rust backend
│   └── deployment configuration
├── PostgreSQL data
├── Private file storage
├── Temporary upload area
├── Logs
└── Backup staging area
```

- Uploaded files के लिए server-generated storage keys और private directories इस्तेमाल करें।
- Temporary uploads और completed files अलग रखें।
- Temporary upload cleanup, deleted-file retention और log rotation के लिए scheduled jobs हों।
- File upload से पहले available disk space और project quota check करें।
- Disk pressure पर uploads को नियंत्रित तरीके से reject/pause करें; disk full होने तक लिखते न रहें।
- PostgreSQL data और file storage की capacity अलग monitor करें।

महत्वपूर्ण: एक ही VPS disk पर backup रखने से disk failure के खिलाफ पर्याप्त protection नहीं मिलता। Backup की अलग storage location रखें।

3. Backup and restore

Backup में दो अलग data classes शामिल होंगी:

1. **PostgreSQL:** users, projects, connection profiles, file metadata, permissions और audit records।
2. **File storage:** actual uploaded images, videos और documents।

Backup policy

- PostgreSQL के लिए consistent database backups और आवश्यकतानुसार point-in-time recovery strategy।
- File storage के लिए incremental या snapshot-based backups, storage provider की capabilities के अनुसार।
- Backup encryption और restricted access।
- Backup retention policy configurable हो।
- Backup jobs की success/failure status और last successful backup track हो।
- Backup completion के साथ checksum/manifest verify करें।

Restore requirements

Restore procedure को नियमित रूप से test करें। Database metadata और files को compatible recovery point से restore करना जरूरी है; अन्यथा metadata ऐसी file की ओर point कर सकती है जो restore नहीं हुई, या उल्टा।

4. HTTPS and network exposure

- Reverse proxy TLS termination और certificate renewal संभाले।
- HTTP requests को HTTPS पर redirect करें।
- Backend service को public interface पर expose न करें; reverse proxy से private/local interface पर traffic लें।
- PostgreSQL port public exposure से बचाएं; remote DB access के लिए allowlisted network, TLS और least-privilege credentials रखें।
- SSH access keys और restricted accounts से हो; password-based public SSH access को deployment policy के अनुसार बंद किया जा सकता है।
- Firewall में केवल आवश्यक inbound ports खोलें।
- TLS certificates, SSH keys और application secrets को source code या ordinary config files में commit न करें।

5. Deployment and operations

- Rust backend को service manager या container runtime के जरिए run करें।
- Production config, secrets और persistent data को application release files से अलग रखें।
- Deployment से पहले configuration validation और database migration check करें।
- Graceful shutdown में नए requests रोकें, active operations को drain करें और फिर process बंद करें।
- Health/readiness endpoints रखें, लेकिन उनमें secrets या sensitive infrastructure details न लौटाएं।
- Logs में request IDs और sanitized errors रखें; tokens, passwords और signed URLs redact हों।
- Rollback procedure और deployment version tracking रखें।

6. Operational monitoring

कम-से-कम ये metrics monitor हों:

- Disk usage और free space
- PostgreSQL availability और storage growth
- File storage size और upload failures
- CPU, memory और process health

- Backup age, duration और failure count
- HTTPS certificate expiry
- API error rate और latency

Part 5 implementation order

1. VPS hardening, firewall और private service networking।
2. Reverse proxy और HTTPS।
3. Rust backend service deployment।
4. PostgreSQL persistence और private file-storage directories।
5. Backup/restore automation और restore drill।
6. Disk, service, certificate और backup monitoring।

Part 6 — Realtime, Performance & Scaling

1. Realtime architecture

Realtime का उपयोग UI को upload progress, connection status और file processing updates देने के लिए होगा।

```
Rust Backend
   ├── Upload / Connection / File Events
   ├── Event Publisher
   └── WebSocket Gateway
            |
            ▼
       Slint Desktop Client
```

- REST API commands और resource operations के लिए primary interface रहेगी।
- WebSocket केवल live status/events के लिए होगा।
- Client reconnect होने पर current status REST API से फिर fetch करे; WebSocket events को permanent source of truth न मानें।
- Events में project/resource scope verify हो; दूसरे project के events client तक न पहुंचें।
- Reconnect के बाद missed events के लिए sequence ID या state refresh mechanism रखें।

उदाहरण event types:

```
upload.progress
upload.completed
upload.failed
connection.state_changed
file.validation_completed
```

2. Upload speed

Upload throughput को client, network, backend और storage—चारों की सीमाएं प्रभावित करती हैं।

Design

- Small files के लिए straightforward streamed upload।
- Large files के लिए resumable/chunked upload।
- Chunks को temporary storage में रखें और completion पर final file commit करें।
- Concurrent uploads पर per-user और per-project limits लागू करें।
- Upload progress को WebSocket या periodic status API से report करें।
- Hash/checksum verification से uploaded content की integrity check करें।

Chunk size, parallelism और concurrent-upload limits को real workloads पर benchmark करें। हर VPS, network और client के लिए एक ही fixed configuration सही नहीं होगी।

3. Large video handling

बड़े videos के लिए पूरी file को memory में load न करें।

- Upload और download streaming-based हों।
- Temporary disk usage और incomplete-upload cleanup नियंत्रित हों।
- Download में HTTP range requests support करने से client resume/seek कर सकता है—इसे storage adapter और response layer में implement करना होगा।
- File size, upload duration और per-project storage quota configurable हों।
- वीडियो का thumbnail/transcoding जैसे processing tasks हों तो उन्हें background worker में रखें; API request को लंबे समय तक block न करें।
- Transcoding को optional future capability रखें; मूल file upload/download के लिए इसकी जरूरत नहीं है।

4. Caching

Caching को correctness और privacy के अधीन रखें।

- Project, connection और file metadata के लिए short-lived in-memory cache उपयोगी हो सकती है।
- Permission-sensitive data को cache करते समय project/user scope key का हिस्सा हो।
- Membership या permission बदलने पर संबंधित cache invalidate हो।
- File content caching private files के लिए default रूप से बंद रखें, जब तक authorization-aware cache policy न हो।
- Cache miss या cache failure पर authoritative PostgreSQL/storage source से data पढ़ा जाए।

Cache authorization का विकल्प नहीं है। हर sensitive request पर permission enforce होनी चाहिए, चाहे metadata cached हो।

5. Performance controls

- File streaming में bounded buffers रखें; पूरी file memory में buffer न करें।
- Slow clients के लिए backpressure लागू करें।
- Upload/download concurrency और per-project bandwidth limits configurable रखें।
- Database queries में pagination और result-size limits लागू हों।
- Pool budgets Part 1/Connection Manager की global और per-server limits के अनुरूप रहें।
- CPU-heavy file inspection या video processing को bounded background workers में चलाएं।
- API request timeout, upload-session expiry और cancellation handling रखें।

6. Scaling strategy

पहले एक VPS पर modular monolith रखें। Scale करने से पहले metrics से bottleneck पहचानें।

Stage	Architecture
Initial	Single Rust backend, PostgreSQL और private VPS disk
Growth	Bounded background workers, stronger monitoring, external backup storage
Higher upload volume	Object-storage adapter और resumable uploads
Multi-instance backend	Shared event delivery, shared session/state strategy और external file storage

एक से अधिक backend instances चलाने पर local in-memory WebSocket event bus पर्याप्त नहीं रहेगा। तब shared pub/sub या event broker की जरूरत पड़ सकती है। उसी तरह, multiple backend instances के लिए local-only file storage के बजाय shared/object storage अधिक उपयुक्त होगा।

7. Failure handling

- WebSocket disconnect होने पर REST API काम करती रहे।
- Upload interruption पर session resume या clean restart हो।
- Disk full या quota exceeded होने पर upload स्पष्ट error के साथ रुके।
- Storage write सफल लेकिन metadata update fail हो जाए, तो reconciliation/cleanup job orphaned files खोजे।
- Metadata commit हो जाए लेकिन file write fail हो, तो file को `ready` न mark करें।
- Background worker restart के बाद incomplete jobs को सुरक्षित तरीके से resume या fail कर सके।

Part 6 implementation order

1. Streaming upload/download और bounded memory use।
2. Upload status API और WebSocket events।
3. Resumable upload sessions और cleanup।
4. Large-file download range support।
5. Scoped metadata caching और invalidation।
6. Background processing, metrics और load testing।
7. जरूरत पड़ने पर shared event delivery और object storage।

Combined final direction

- **Part 5:** VPS, HTTPS, private networking, disk layout, backups और operational monitoring।
- **Part 6:** WebSocket status updates, resumable streaming, large-video handling, scoped caching और measured scaling।
- प्रारंभिक deployment single VPS पर रहे; object storage, multi-instance backend और event broker को वास्तविक load या availability requirements आने पर जोड़ा जाए।